

2024 Sep Oppe 1 - Set 1

Section 3 - Problem 1

Print Pattern - V

Given an integer n (where $n > 0$), print a "V" shaped pattern with n rows. The pattern should use backslashes (\) and forward slashes (/) for each row, with the v character at the bottom. There should be no spaces to the right of the pattern.

NOTE: This is an I/O type question, you need to write the whole code for taking input and printing the output.

Input Format

- A single integer n , representing the number of rows in the pattern.

Output Format

- A "V" shaped pattern with n rows, as described.

Constraints

- $1 \leq n$

The Specification

INPUT:

A single integer **n** (representing total rows).

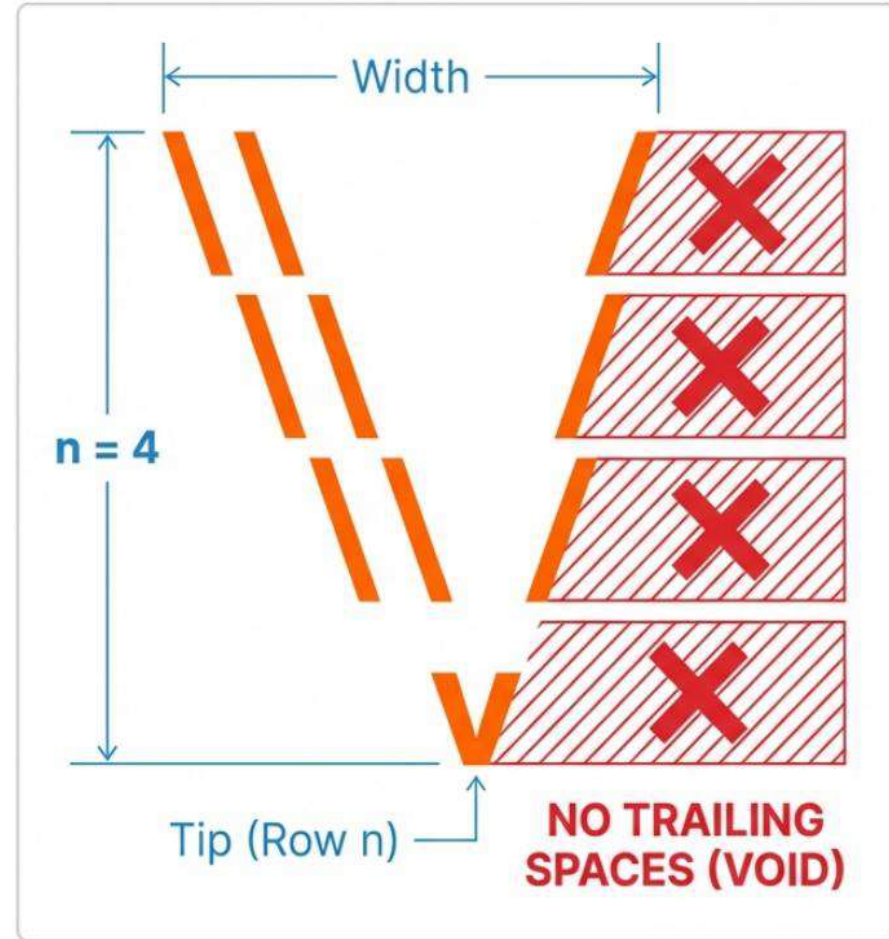
CHARACTERS:

1. **Left Leg:** Backslash `\` (Requires escape character `\\`)
2. **Right Leg:** Forward slash `/`
3. **Tip:** Lowercase `v` (Last row only)

CRITICAL CONSTRAINT:

No trailing spaces. Automated tests will fail if invisible whitespace exists to the right of the last visible character on any line.

V-Pattern Diagram (n=4)



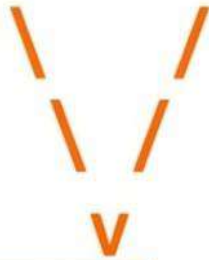
Visualizing the Progression

Observe how the pattern scales with input n .

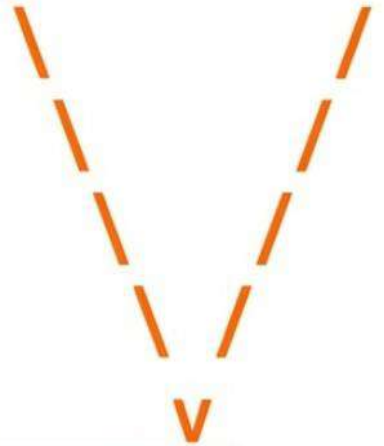
Case A: $n=1$



Case B: $n=2$



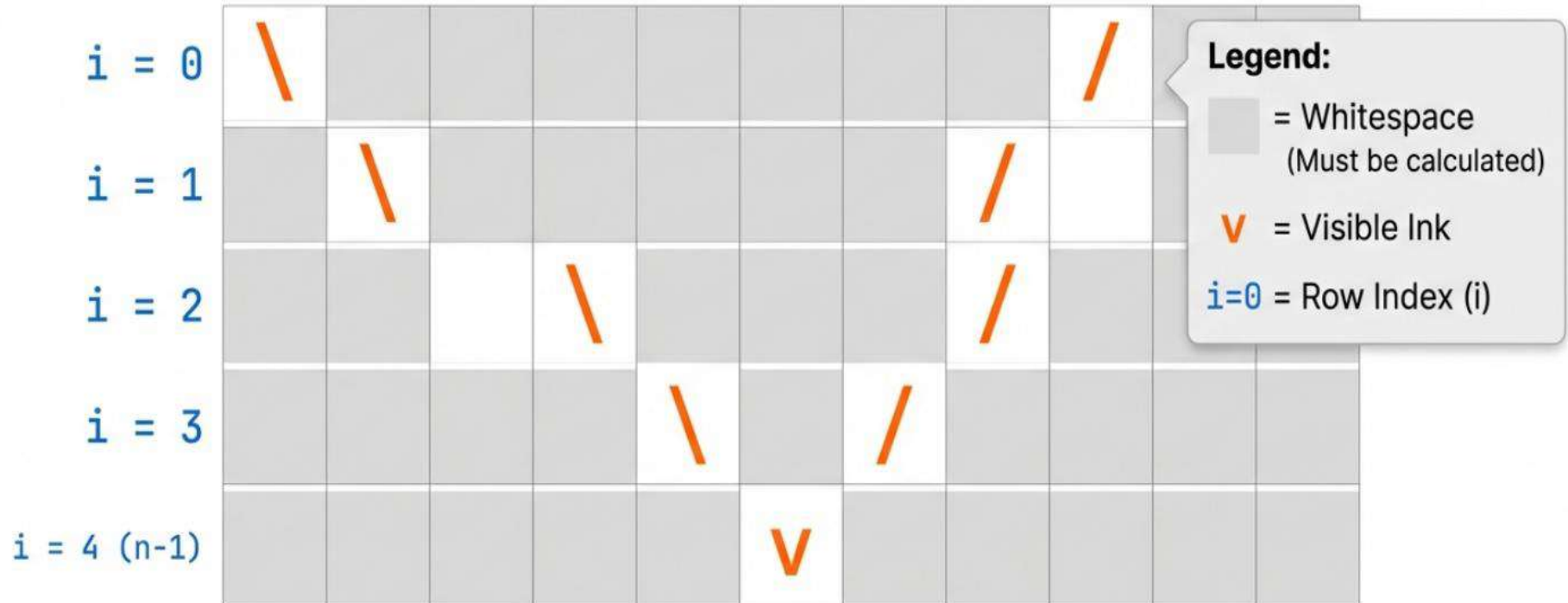
Case C: $n=5$



Observation: As n increases, the legs lengthen, but the single 'v' tip remains the constant anchor at the bottom.

The Invisible Grid

Code **prints** lines of characters. We must calculate the **coordinates**.



Component 1: The Leading Spaces

Leading Spaces = i

Row (i)	Spaces
0	0
1	1
2	2
3	3

Insight: The indentation increases linearly. Row 0 has 0 spaces. Row 3 has 3 spaces.

$i = 0$



$i = 1$



$i = 2$



$i = 3$



$i = 4 \text{ (n-1)}$



Component 2: The Middle Gap

The Math

$$\text{Gap} = 2 * (n - i - 1) - 1$$

Remaining rows
below current.

Two width units
lost per row
(left & right).

Zero-index
adjustment.

Example calculation trace for $n=5, i=0$:

$$\text{Gap} = 2 * (5 - 0 - 1) - 1$$

$$\text{Gap} = 2 * (4) - 1$$

$$\text{Gap} = 7 \text{ spaces}$$

The Visual

$i = 0$

$i = 1$

$i = 2$

$i = 3$

$i = 4 \text{ (} n-1 \text{)}$

		7 spaces		/	
\		5 spaces	/		
\		3 spaces	/		
	\	1 space	/		
		V			

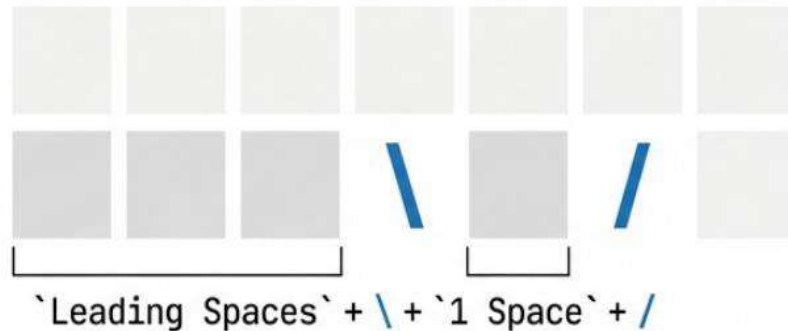
The Edge Case: The Tip

Where the pattern breaks.

Standard
Logic

Zoom In: Last Two Rows (n=5)

i = 3
Row n-2



Edge Case
(i == n - 1)

i = 4
Row n-1



The Conflict

Gap Formula at i=n-1:

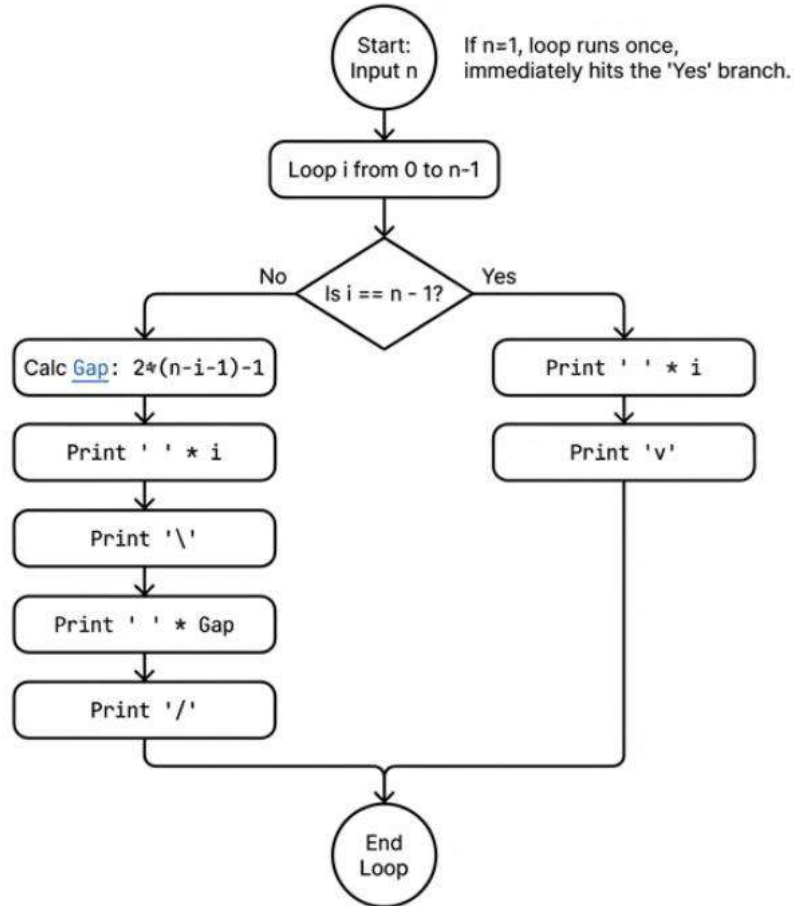
$$2 * (5 - 4 - 1) - 1 = -1$$

****Error: Cannot print negative spaces.**

The Resolution

Override: If last row, print 'v' only.

The Logic Flow



Approach 1: The Explicit Solution

```
# Manual Loop Approach
```

```
if n == 1:
```

```
    print("v")
```

```
else:
```

```
    for i in range(n - 1):
```

```
        leading = " " * i
```

```
        gap = " " * (2 * (n - i - 1) - 1)
```

```
        print(f"{leading}\{gap}/")
```

```
# Print tip separately
```

```
print(" " * (n - 1) + "v")
```

Loop stops before the last row.

Complex logic isolated here.

The tip is handled outside the loop.

Pros: Easy to debug step-by-step.

Cons: Logic is split into two places.

Approach 2: The Elegant Refactor

Leveraging string multiplication for cleaner code.

```
for i in range(n):  
    # 1. Leading spaces  
    spaces = " " * i
```

Python Feature: String Multiplication eliminates inner loops.

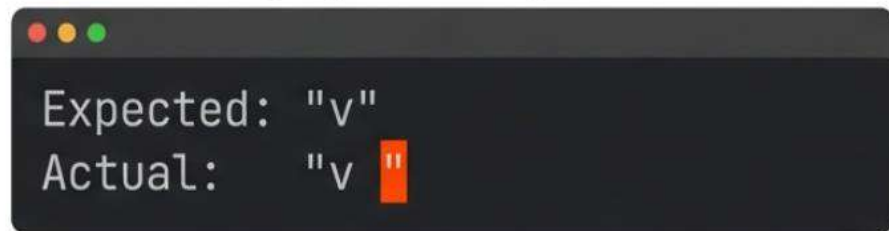
```
    # 2. Check for Tip  
    if i == n - 1:  
        print(spaces + "v")
```

```
    # 3. Standard Legs  
    else:  
        gap = " " * (2 * n - 2 * i - 3)  
        print(spaces + "\\\" + gap + "/"
```

Unified Logic: Single loop handles both cases.

Technical Gotchas & Best Practices

The Trailing Space Trap



Do not print spaces after the last character. Tests will fail.

Coordinate Mapping

i vs **count** ← Use for math & clarity

Loop row index Manual counter variable

Use the row index `i` for math. Avoid manually incrementing separate counter variables.

Escaping Characters

```
print("\") → **Error**  
print("\\") → \ (Success)
```

The backslash is a special escape character. You must double it.

Range Boundaries

range(n)
[0 ... n-1]
Includes n elements

range(n-1)
[0 ... n-2]
Stops before last

Be explicit about whether your loop includes the tip row or stops before it.